

Principled approaches for extending neural architectures to function spaces for operator learning

Received: 30 July 2025

Accepted: 5 June 2026

Published online: 03 July 2026

 Check for updates

Julius Berner^{1,3}, Miguel Liu-Schiaffini^{2,3}, Jean Kossaifi¹, Valentin Duruisseaux^{1,2}, Boris Bonev¹, Kamyar Azizzadenesheli¹ & Anima Anandkumar²✉

Deep learning has achieved remarkable success in computer vision and natural language processing, where tasks are commonly formulated as mappings between finite-dimensional representations. Many scientific problems, however, including those governed by partial differential equations, are naturally posed on infinite-dimensional function spaces. This mismatch has limited conventional neural networks from achieving comparable success in scientific applications. Here we identify and distil key principles for constructing practical neural architectures for mappings between function spaces. Neural operators provide a principled extension of neural networks to such settings, offering a path towards bringing deep learning's transformative impact to science. Because deep learning's success has relied heavily on architectural refinements, extending these advances to neural operators allows operator learning to benefit from refined designs. Guided by the principles we outline, we propose a recipe for converting popular neural architectures into neural operators with minimal modifications. We also discuss practical steps for making these models effective. This perspective offers a systematic bridge between finite-dimensional network design and operator learning for scientific applications.

In the past decade, deep learning has led to unprecedented advances in domains such as computer vision, speech and natural language processing. Following this success, deep learning is poised to have an arguably even greater transformational impact on the natural sciences. Although underlying data in all these areas come in vastly different structures, the standard models powering deep learning, neural networks, consume processed versions of data in the form of finite-dimensional vectors (for example, pixel representations of images and videos, word embeddings of language or measurements of physical systems).

Yet, many physical phenomena of interest are inherently captured by continuum descriptions: quantities in fluid dynamics and thermodynamics, continuum mechanics or electromagnetism depend on continuous spatiotemporal variables and are often governed by

partial differential equations (PDEs), where the central task is to map an input function, such as coefficients or forcing terms, to a solution function. Common tasks in computer vision can be viewed similarly: natural scenes or objects can be idealized as functions of space and time, whereas images and videos are finite-resolution discretizations of underlying functions.

Although classical neural networks can process discretizations of continuous quantities (for example, on a mesh or grid), they typically either rely on a fixed discretization or do not generalize to unseen resolutions. However, we would like the learned map to be discretization agnostic with respect to the specific choice of discretization and its resolution (up to a vanishing discretization error). Furthermore, in numerous applications, it is essential for the output to be a function

¹NVIDIA Corporation, Santa Clara, CA, USA. ²Department of Computing and Mathematical Sciences, California Institute of Technology, Pasadena, CA, USA.

³These authors contributed equally: Julius Berner, Miguel Liu-Schiaffini. ✉e-mail: animakumar@gmail.com

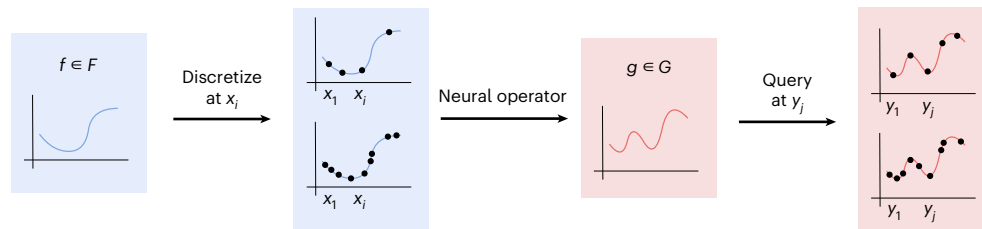


Fig. 1 | Illustration of a neural operator. The input is a function $f \in \mathcal{F}$ that can be given at any discretization $(x_i)_{i=1}^n$. The output is a function $g \in \mathcal{G}$ that can be queried on any discretization $(y_j)_{j=1}^m$.

that can be queried at arbitrary coordinates, enabling further downstream operations (for example, differentiation and integration). To this end, neural networks have recently been generalized to neural operators, which are naturally formulated to operate on functions rather than vectors (Fig. 1). The empirical neural operator is evaluated on samples, but its parameters describe a function-space map: by construction, it outputs functions or empirical evaluations of functions that can be queried at arbitrary coordinates and that are consistent across different discretizations of the input function (more precisely the outputs only differ by a discretization error that vanishes as the discretization is refined). This property of being agnostic to the underlying discretization enables learning of function-to-function mappings and multiscale phenomena for the wide range of tasks where the underlying data are captured by continuum descriptions. These properties have led to substantially improved performance and generalization, as well as powerful capabilities, such as zero-shot super-resolution, in a series of practical applications. Supplementary Sections A.2, A.3 and A.5 formalize the empirical evaluation of neural operators on discretized inputs, define discretization convergence and relate discretization, approximation and optimization errors to zero-shot prediction across resolutions.

Using neural networks to represent continuous functions, known as implicit neural representations or neural fields, has a long history¹. Special cases include physics-informed neural networks², where the neural network approximates a PDE solution function by optimizing its parameters to reduce deviations from the governing equations. Shortly after the introduction of neural operators, three-dimensional scenes in graphics and computer vision have been similarly represented using neural radiance fields^{3–5}. In both of these cases, the neural network represents a single function, that is, a single PDE solution or three-dimensional scene. By contrast, neural operators can be viewed as generalizations of neural fields because they output a function for any given input function, that is, they can learn operators that act on functions. Neural operators can be viewed as conditional neural fields⁶, where the conditioning variable is the input function. Alternatively, one can condition the implicit neural representations on the parameters of a finite-dimensional parametrization of the space of input functions^{7,8}. Although this can be viewed as a special case of an encoder–decoder neural operator (Supplementary Section B.6), neural operators are more general and can handle input and output functions based on point evaluations without the need for parametrizations.

In this Article, we present how neural network layers can often be extended to function spaces by identifying the continuous operator that the finite-dimensional layer discretizes. Once that operator has been identified, the layer can be discretized in a way that respects the coordinates, quadrature weights and sampling structure of the underlying function, rather than only the index set of a particular array.

We first describe the operator-learning setting and the principles that make neural operators discretization agnostic. We then give a general recipe for converting neural network layers into operator layers, summarize the resulting correspondences for fully connected layers, pointwise maps, convolutions, graph neural networks,

transformers and encoder–decoder architectures and finally evaluate these distinctions empirically.

From discrete to continuous learning on function spaces

Classical neural networks aim to approximate functions

$$x \mapsto f(x) \quad (1)$$

between (finite-dimensional) Euclidean spaces, such as $\mathbb{R}^d$. More precisely, they take a vector $\mathbf{x}$ as input and output another vector $f(\mathbf{x})$. By contrast, we want to construct operators

$$f \mapsto g \quad (2)$$

between (infinite-dimensional) function spaces, such as continuous functions. We use $\mathcal{F}$ and $\mathcal{G}$ to denote the function spaces for the input and output functions, which are typically suitable subspaces of continuous functions. Operators take a function f as input and output another function g (see Table 1 for a comparison and Supplementary Section A.1 for details on notation).

The transition from discrete to continuous problems requires a change in how we represent and manipulate data. Although we are interested in learning mappings between functions, we typically do not have access to the actual functions numerically. Instead, neural operators are designed to work on a discretization $((x_i, f(x_i)))_{i=1}^n$ of a function f , assuming that we observe the function on a set of points x_i (that is, a point cloud or grid; see Supplementary Section A.1 for alternative representations). Moreover, the output of the neural operator, a function g , can be queried at arbitrary coordinates y in its domain (see Fig. 1 for an illustration).

A map learned directly between two arrays at one resolution need not define a consistent map between the underlying functions. If the parameters, receptive field, aggregation rule or tokenization are tied to input indices rather than domain coordinates, changing the discretization changes the operation applied. The goal is therefore not merely to accept arrays of varying sizes but to ensure that these arrays are all discretizations of the same underlying function-space map.

We now formulate a collection of practical principles for the design of neural operators to guide their effective use in scientific machine learning.

Defining principles of neural operators

To learn a mapping between functions in practice, the following properties should be satisfied:

- **Discretization agnostic:** the learned mapping should be applicable to functions given at any discretization and produce consistent outputs across resolutions.
- **Fixed number of parameters:** the number of learnable parameters needs to be fixed and independent of the discretization.
- **Universal approximation:** the family of mappings should be able to (provably) approximate, with arbitrarily low error, any sufficiently regular operator.

Table 1 | Notation and comparison of neural networks to neural operators

	Neural network	Neural operator	Input	Output
Space	Euclidean spaces (for example, $\mathbb{R}^d$)	Function spaces	$\mathcal{F}$	$\mathcal{G}$
Elements	Vectors	Functions	$f \in \mathcal{F}$	$g \in \mathcal{G}$
Domain	Integer indices	Real-valued coordinates	$x \in D_f$	$y \in D_g$

Neural operators represent a principled extension of neural networks to function spaces designed from first principles to satisfy these properties. They are parametrized by a finite number of learnable parameters, universally approximate sufficiently regular function-to-function mappings and provide consistent predictions across resolutions. In practice, neural operators still operate on discretized functions but also respect the underlying functions they model and are not tied architecturally to a fixed set of observation points. In particular, their outputs are functions that can be evaluated on arbitrary discretizations, and the outputs of a neural operator for different discretizations only differ by a discretization error that can be made arbitrarily small by refining the discretization, in which case we say that the neural operator is discretization convergent (Supplementary Section A.3).

Features of neural operators

Neural operators provide a principled framework to learn mappings on functional data, and their ability to train and test on data with varying discretizations unlocks unique features and advantages:

- **Canonical function-space formulation:** for many tasks, the ground truth mapping is naturally an operator, that is, a mapping between functions. If we train a neural network that is not discretization agnostic, we can overfit to the training discretization(s). Operator learning allows us to approximate the ground truth operator and establish rigorous theoretical guarantees (Supplementary Section A.5). In addition, having output functions that can be queried arbitrarily is useful for downstream applications that require derived quantities such as derivatives or integrals.
- **Data efficiency:** standard numerical solvers often create their meshes adaptively, which naturally leads to datasets with different discretizations^{9–11}. Due to high computational costs, only a few high-resolution solutions are available in many applications, whereas a larger number of approximate solutions can be computed on discretizations with lower resolutions. Operator learning enables learning on different discretizations and resolutions simultaneously, ensuring sufficient data for training while maintaining the ability to resolve the underlying physics from the data at higher resolution. In practice, neural operators can work directly with functions discretized on point clouds with an arbitrary number of points and do not need an explicit mesh or element partition of the domain.
- **Curriculum learning and faster training:** neural operators can be trained via curriculum learning, where simpler, low-resolution samples are first used for training, gradually progressing to more challenging, high-resolution samples. Such adaptive refinement of the discretization during training can not only improve performance but also accelerate convergence and reduce the computational cost compared with training only on high-resolution data^{12,13}.
- **Flexible inference:** a trained neural operator can be queried at arbitrary resolution. For instance, we can obtain consistent predictions for discretizations that have not been in the training data, both at lower and higher resolutions, that is, zero-shot super-resolution.

The desired principles and unique advantages of neural operators have spurred considerable interest in developing architectures and

dedicated Python libraries, such as the NEURALOPERATOR library¹⁴, and their use in numerous scientific applications (Supplementary Section E).

Motivated by the properties and capabilities of neural operators, we next discuss how to construct such architectures.

A recipe for moving from networks to operators

Although there is no universal automated process to generalize arbitrary neural network layers and architectures to function spaces, there is a consistent underlying strategy that motivates and informs all the examples we showcase. As summarized in Fig. 2, the first step is to investigate the neural network architecture to understand what would be an analogous transformation in terms of the continuous coordinates of the domain and the functions of interest (that is, which continuous operator the layer discretizes and tries to approximate). Building on this, the next step is to design strategies to discretize that continuous transformation in a way that respects the core principles presented above.

In practice, more precise mechanisms that follow this overall strategy are shared by families of layers and architectures. As an example, many neural network architectures can be framed as variants of graph neural networks

$$\mathbf{g}_j = \sum_{i \in \text{Neigh}_j} K_{ij}(\mathbf{f}_i, \mathbf{f}_j). \quad (3)$$

The common strategy we follow for these neural network architectures is to use the explicit positional information (that is, the coordinates x on the domain of f) and replace the message-passing summation over neighbours by an integral over a suitable neighbourhood $D(y_j)$ in the underlying domain

$$g(y_j) = \int_{x \in D(y_j)} K(x, y_j, f(x), f(y_j)) \, dx. \quad (4)$$

This integral operator is analogous to the graph neural network, in terms of the continuous coordinates x and the relevant functions K and f . In practice, the integral has to be discretized, which can be achieved by aggregating the contributions in each neighbourhood using appropriate quadrature weights Δ_i (Fig. 5), thereby leading to the graph neural operator

$$g(y_j) = \sum_{i: x_i \in D(y_j)} K(x_i, y_j, f(x_i), f(y_j)) \Delta_i, \quad (5)$$

which respects the core principles we outlined and enjoys the resulting desirable properties. If y_j is only a query coordinate, $f(y_j)$ is not assumed available; the kernel can instead use $(x_i, y_j, f(x_i))$ together with query information such as the coordinate y_j , a positional encoding or an interpolated value, when appropriate.

At a high level, this recipe extends multilayer perceptrons, convolutional neural networks, graph neural networks and transformers to integral, convolutional, graph and attention-based operator layers. Other components require different modifications: encoders and decoders need discretization-agnostic latent interfaces, normalization must compute statistics as function-space quantities and point-wise operations such as activations, liftings and projections transfer almost directly.

Architecture correspondences

The recipe above produces concrete correspondences between familiar neural network layers and their function-space analogues. The unifying message is that successful finite-dimensional inductive biases should first be expressed as transformations of functions and coordinates and only then discretized.

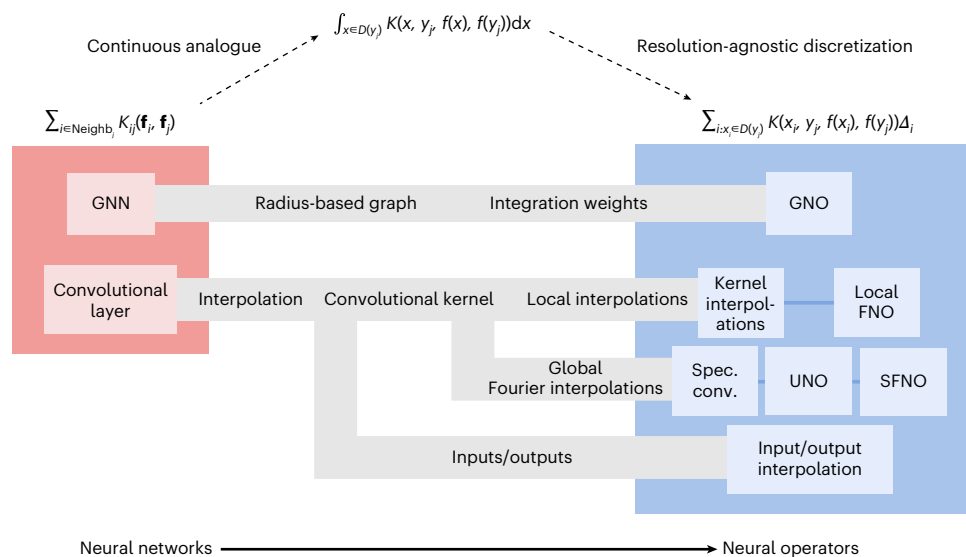


Fig. 2 | Pipeline of converting neural networks to neural operators. Graph neural network (GNN) and convolutional layers can be converted into well-posed neural operator layers through a sequence of simple modifications. GNO refers to

the graph neural operator²³, ‘Spec. conv.’ refers to a spectral convolution as used in FNOs^{15,16}, and local FNO refers to an FNO supplemented with local integral kernels¹⁸. We denote by UNO the U-shaped neural operator²⁴ and by SFNO the spherical FNO²⁵.

Fully connected layers become integral transforms. The fully connected layer is the basic building block of multilayer perceptrons and neural networks. The affine-linear map assumes a fixed, finite-dimensional vector as input, with weights indexed by input and output indices. To make the same operation act on functions, the entries of the weight matrix and bias vector are replaced by learnable functions of input and query coordinates, and the discrete sum is interpreted as a quadrature approximation. This naturally extends a fully connected layer from an affine-linear map between vectors to an affine-linear integral transform between functions.

Elementwise functions become pointwise operators. Several common layers in deep learning apply the same operation to each element of the input vector, such as activation functions, (1×1) convolutions or certain normalization layers. They can be used directly in neural operators if these elements correspond to pointwise evaluations of the input function. Because no spatial aggregation is involved, pointwise operators are naturally agnostic to the number of samples and provide activations, skip-connections, liftings and projections for neural operators. Their limitation is that the output is initially available at the same coordinates as the input; querying at new points requires interpolation or composition with another operator layer.

Convolutional layers become convolutional or spectral operators. A discrete convolution layer can be applied to grids of different sizes, but its physical receptive field is tied to the grid spacing: a fixed number of neighbouring indices corresponds to different regions of the physical domain at different resolutions. To make the receptive field independent of the discretization, the kernel is defined over physical coordinates with a fixed support radius, as illustrated in Fig. 3. For global periodic convolutions, the same continuous convolution can be implemented efficiently through the convolution theorem, leading to spectral convolution layers and to the Fourier neural operator (FNO)^{15,16}. Thus the receptive field becomes a property of the domain, not of the number of grid points.

Graph neural networks become graph neural operators. Discretizations of functions on more general point clouds can be interpreted as graphs with a given connectivity. A graph neural network aggregates messages over graph neighbourhoods, where the neighbourhood

is often defined by indices or graph edges. A graph neural operator replaces this index-defined aggregation by a quadrature approximation to an integral over a coordinate-defined neighbourhood. Coordinates and quadrature weights are the key additions that make message passing consistent across meshes, point clouds and resolutions. Depending on the neighbourhood, graph neural operators (GNOs) can represent local or global integral operators and subsume the integral-transform and convolutional cases as special choices.

Transformers become transformer neural operators. Transformers can be converted into neural operators using the same principle, with one additional observation: attention is already a global aggregation mechanism, but the continuum limit requires quadrature weights in both the aggregation and normalization. For equal quadrature weights, as on regular grids, the weights cancel and the formula reduces to standard attention. As the attention kernel acts on values of f , positional information must be supplied through the features, for instance, by concatenating coordinates or using positional encodings. In cross-attention, query features live on the output point cloud and may be values of another function, coordinates or positional encodings. The resulting layer is a global, input-dependent integral operator. It does not impose translation equivariance, whereas FNOs can be understood as structured special cases when translation equivariance is appropriate and the kernel is parametrized in the Fourier domain.

Encoders and decoders become interfaces between functions and latent variables. Encoders and decoders are commonly used so that the core neural network operates in a latent space. Such an idea can also be used to construct neural operator layers, but the encoder must be agnostic to the discretization of the input function and the decoder must produce a function that can be queried at arbitrary resolution. One representative construction encodes the input through inner products with learned functions, applies a finite-dimensional map in latent space and decodes at query coordinates. The latent dimension is fixed independently of the number of input samples, which makes the interface discretization agnostic. Encoder–decoder operators are universal in principle, but a fixed latent bottleneck can limit high-resolution information. This perspective also clarifies why the interpolation-based baselines we consider are limited neural operators.

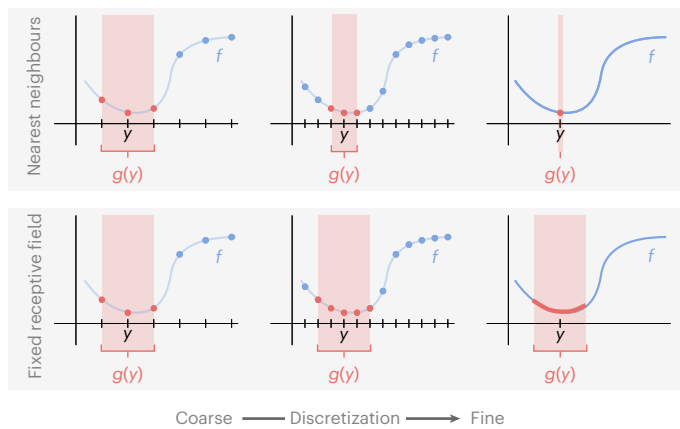


Fig. 3 | Illustration of collapsing receptive fields with a nearest neighbours strategy. The figure shows the values of the input function f (blue) that influence the output function g at a point y when using a nearest neighbours strategy (for example, as in convolutional and graph neural networks) (top) and with a fixed receptive field (for example as in convolutional and graph neural operators) (bottom). If the neighbourhood is selected using a nearest neighbours strategy, the receptive field (red) collapses when the discretization is refined (from left to right).

Taken together, these correspondences show that the operator viewpoint does not discard the inductive biases of modern neural networks. It instead asks that each bias be expressed at the level of coordinates and functions before being discretized.

Empirical evaluations

We empirically evaluate different architectures to quantify the difference between neural networks and neural operators and to test whether the proposed recipes improve generalization across resolutions. The main comparison uses the Navier–Stokes equations for incompressible fluids, mapping a forcing function to the vorticity at a later time from zero initial vorticity. Methods summarize the experimental setup, with further details in Supplementary Sections F.1–F.3.

Figure 4 shows that architectures whose operations are defined at the level of the domain, such as FNO and OFORMER, generalize across resolutions. Fixed-discretization neural networks such as U-Net and ViT degrade away from the training resolution because their receptive fields or tokenization depend on the grid: when neighbouring points move closer or farther apart in the physical domain, the layer implements a different operation. The operator modifications address this issue in complementary ways. Kernel interpolation makes a convolutional neural network (CNN) less tied to a single grid but can still suffer from discretization error when the learned kernel is low resolution. Spectral convolution in the FNO provides strong performance across all considered resolutions. Global attention, as used in OFORMER¹⁷, also generalizes well, although its computational cost becomes large at high resolution.

Training U-Nets on multiple resolutions leads to good performance at those resolutions but still poor generalization to unseen resolutions. Such augmentation encourages consistency on the finite set of discretizations observed during training, but it does not by itself provide a principled inductive bias for unseen resolutions. Similarly, interpolation-based encoder–decoder baselines define neural operators with fixed encoders and decoders, but their performance is limited because high-resolution information can be discarded before the learnable component is applied.

Composite architectures inherit both the benefits and discretization errors of their components. The FNO + LOCALCONV and FNO + DIFFCONV variants combine spectral convolution with local operator layers¹⁸, illustrating not only that local and global receptive fields can be useful in combination but also that every component has a resolution range in which its discretization error is controllable.

In summary, our experiments support the conclusion that cross-resolution generalization depends on domain-level receptive fields, principled discretization and preservation of high-resolution information. Multiresolution training can improve performance on seen resolutions, and interpolation can define a naive neural operator, but robust generalization to unseen resolutions is best supported when the architecture itself is constructed as a discretization of a well-defined function-space operation.

Discussion

We show how simple modifications can make popular neural network architectures agnostic to the discretization, giving rise to neural operators as a principled framework for learning mappings between function spaces. We defined the general design principles and demonstrated how discretization-agnostic versions of CNNs, graph neural networks, transformers and encoder–decoder architectures give rise to general building blocks for neural operators.

The practical implication is that operator learning does not need to be separated from progress in architectural design. Popular architectures can be extended to function spaces by replacing index-based operations with coordinate-aware continuous operators, by introducing quadrature weights where sums approximate integrals and by ensuring that latent interfaces are independent of the discretization used to observe the input. This framing is useful beyond the benchmark PDEs used to illustrate the ideas: neural operators have already been applied across a large variety of applications. The common thread is that inputs and outputs are naturally fields, whereas practical data are observed on meshes, grids or point clouds that may vary across simulations, instruments or resolutions.

Because neural operator design is still an emerging field, many research directions remain open. For architectures satisfying discretization convergence, the discretization error vanishes as the resolution is increased; nevertheless, it is important to be able to quantify and control this error in practice. Discretization-agnostic designs can sometimes come at the cost of reduced performance at a fixed resolution compared with specialized neural network architectures. This is not a fundamental shortcoming of neural operators but rather requires further development of architectures with the right inductive biases. As starting points, one can leverage connections with neural operators to successfully use techniques from computer vision, such as implicit neural representations or incorporate ideas from classical numerical methods. Additional important open directions include stronger discretization-error estimates for practical models, operator architectures for complex geometries and adaptive discretizations, efficient attention mechanisms at high resolution, hybrid local–global operators and theory that better predicts when architectural choices transfer across resolutions.

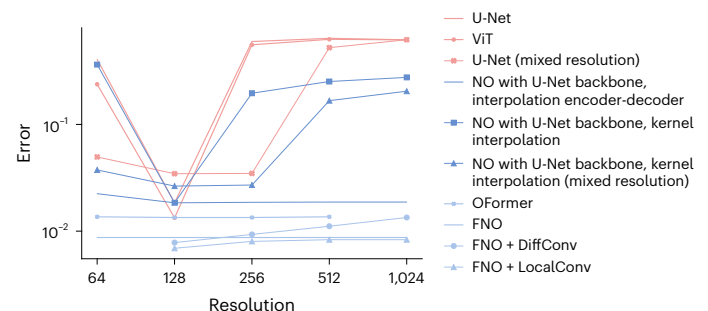


Fig. 4 | Relative L^2 errors on unseen Navier–Stokes test data. FNO and OFORMER, trained only at a resolution of 128, generalize across lower and higher resolutions. By contrast, the U-Net and ViT degrade away from the training resolution. Mixed-resolution training helps on seen resolutions, whereas kernel interpolation improves generalization relative to the baseline U-Net.

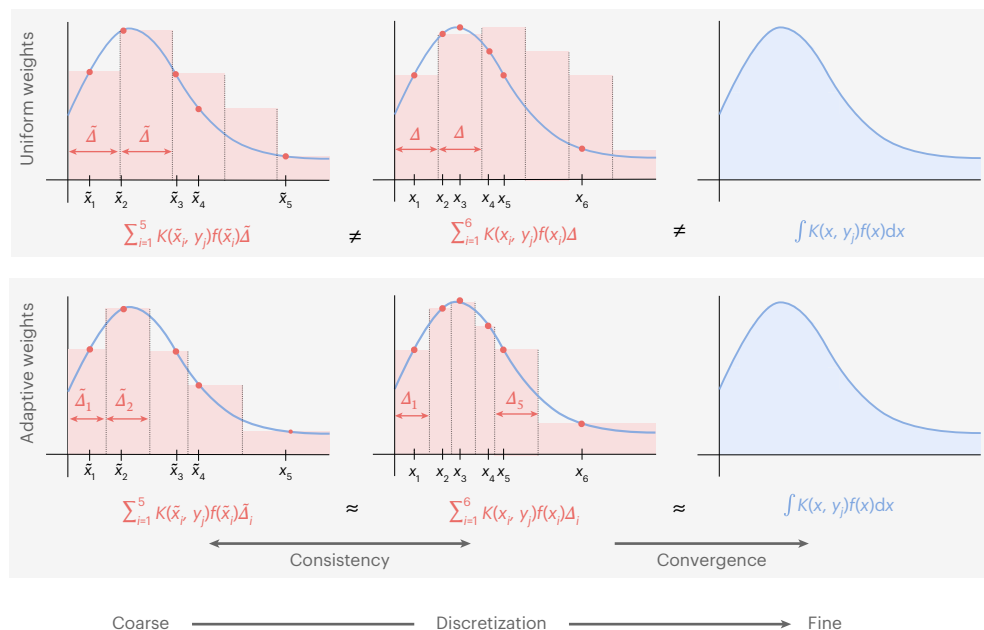


Fig. 5 | Visualizing the need for quadrature weights. Aggregating function values at irregularly spaced points without proper quadrature weights (for example, taking the mean by using uniform weights, that is, $\Delta = 1/n$ (top)) adds more weight to densely sampled areas. When increasing the resolution, the output depends on the chosen refinement of the discretization and does not

have a unique limit (top). The use of quadrature weights leads to consistent approximations for different discretizations that converge to a unique integral as the discretization is refined (bottom). Here, the quadrature weight Δ_i associated with x_i is taken to be the length of the segment between the midpoints of its neighbouring points.

Our work provides general guidelines to help practitioners understand the design principles and develop specialized neural operators for their applications. Although neural operators have already seen great success in a variety of applications, we believe that they have the potential to accelerate progress and foster breakthroughs across a wide range of additional fields. Designing in function space lets scientific machine learning inherit deep learning's architectural progress without sacrificing continuum structure.

Methods

Operator-learning setup

Let $\mathcal{F}$ and $\mathcal{G}$ denote the input and output function spaces, with input functions $f \in \mathcal{F}$ defined on a domain D_f and output functions $g \in \mathcal{G}$ defined on a domain D_g . A neural operator is represented by a parametrized map $\text{NO}_\theta : \mathcal{F} \rightarrow \mathcal{G}$, but in experiments, it is implemented through an empirical operator $\tilde{\text{NO}}_\theta$ acting on finite discretizations. Specifically, the input function is observed through samples $f|_X = ((x_i, f(x_i)))_{i=1}^n$ at input coordinates $X = (x_i)_{i=1}^n$, and the empirical operator returns evaluations of the predicted output function at query points $Y = (y_j)_{j=1}^m$. Thus, although the model is evaluated on finite arrays, its parameters are intended to define a single map between function spaces rather than a separate map for each resolution. Discretization-agnostic evaluation refers to the ability to apply the same trained parameters across different input and output discretizations, with predictions that are consistent up to discretization error. Cross-resolution testing assesses this property by training at one resolution or on a finite set of training resolutions and testing on resolutions not used to fit the parameters. Supplementary Section A.1 describes the notation for continuous and discretized functions.

General construction of neural operator layers

Following a common paradigm in deep learning, an effective approach to designing neural operators is to construct simple and modular layers that can be composed to form expressive neural operator models. Each layer is itself an operator such that the intermediate outputs after each layer are functions, as opposed to vectors in standard neural networks.

The first step is to investigate the neural network architecture to understand what would be an analogous transformation in terms of the continuous coordinates of the domain and the functions of interest, that is, which continuous operator the neural network roughly discretizes. Building on this, the next step is to design strategies to discretize that continuous transformation in a way that respects the core principles of neural operators.

Concretely, index-dependent parameters are replaced by coordinate-dependent functions, finite sums over indices are replaced by integrals over the input domain or over coordinate-defined neighbourhoods, and the resulting integrals are evaluated through quadrature on the available discretization. The parameters of the layer therefore define the kernel, pointwise map, attention mechanism, basis functions or latent transformation at the continuum level, rather than entries of a resolution-specific array. In channel-suppressed notation, many of the layers considered below can be viewed as discretizations of transformations of the form

$$g(y) = \Psi(y, f(y)), \int_{D_f} K(x, y, f(x), f(y)) dx, \quad (6)$$

where K and Ψ are learnable functions or compositions of learnable functions. This template covers layers that use both input-side values and query-side features. When y is not an input sample, $f(y)$ is replaced by the query information available to the architecture, such as the coordinate y , a positional encoding, a target-side feature or an interpolated value when appropriate. The precise form changes from one architecture family to another, but the discretized implementation always replaces the integral by a weighted sum over sampled coordinates

$$\int_{D_f} K(x) dx \approx \sum_{i=1}^n K(x_i) \Delta_i. \quad (7)$$

This is the basic mechanism by which the layer can be evaluated on a variable number of input points while preserving a fixed parameter count.

Quadrature weights Δ_i are used whenever the discrete aggregation is intended to approximate an integral over the input domain.

Coordinate information is supplied either explicitly through the kernel arguments, by defining neighbourhoods in the physical domain or through positional encodings. Interpolation is used only as a baseline or auxiliary operation; when applied to a fixed intermediate grid, it defines a naive encoder–decoder operator but may discard information present in higher-resolution inputs.

Architecture-specific operator constructions

The correspondences between neural network layers and neural operator layers are implemented as detailed below. Full derivations of these constructions are collected in Supplementary Sections B.1–B.7, with additional details on FNOs in Supplementary Section C and representative pseudocode in Supplementary Section G.

Integral-transform layers. A fully connected layer maps an input vector to an output vector by an affine-linear transformation

$$\mathbf{g}_j = \sum_{i=1}^n \mathbf{K}_{ji} \mathbf{f}_i + \mathbf{b}_j. \quad (8)$$

To make this operation agnostic to the discretization, we assume that $\mathbf{K}_{ji}$ and $\mathbf{b}_j$ are evaluations of functions K and b at input and output coordinates, with the quadrature weights included in the discretization. This gives the empirical integral-transform layer

$$g(y_j) = \sum_{i=1}^n K(x_i, y_j) f(x_i) \Delta_i + b(y_j) \approx \int_{D_f} K(x, y_j) f(x) dx + b(y_j). \quad (9)$$

The same kernel and bias functions can be evaluated for arbitrary numbers of input and output points, which naturally extends a fully connected layer from an affine-linear map between vectors to an affine-linear integral transform between functions.

Pointwise layers. Several common layers in deep learning apply the same operation to each element of the input vector, such as activation functions, (1×1) convolutions, liftings or projections. They can be used directly in neural operators if these elements correspond to pointwise evaluations of the input function

$$\mathbf{g}_j = K(\mathbf{f}_j) \rightsquigarrow g(x_i) = K(f(x_i)). \quad (10)$$

Because no spatial aggregation is involved, pointwise operators are naturally agnostic to the number of samples. Their limitation is that the output is initially available at the same coordinates as the input; querying at new points requires interpolation or composition with another operator layer.

Convolutional and spectral layers. A discrete convolution uses a fixed stencil in index space

$$\mathbf{g}_j = (\mathbf{K} * \mathbf{f})_j = \sum_{i=j-k}^{j+k} \mathbf{K}_{j-i} \mathbf{f}_i. \quad (11)$$

This operation can be applied to grids of different sizes, but its physical receptive field is tied to the grid spacing. To make the receptive field independent of the discretization, the kernel is defined over physical coordinates with a fixed support radius r

$$g(y_j) = \sum_{i: |x_i - y_j| \leq r} K(y_j - x_i) f(x_i) \Delta_i \approx \int_{y_j-r}^{y_j+r} K(y_j - x) f(x) dx. \quad (12)$$

For global periodic convolutions, the same continuous convolution can be implemented efficiently through the convolution theorem

$$K * f = F^{-1}(F(K)F(f)). \quad (13)$$

This leads to spectral convolution layers, which can be composed to form the FNO^{15,16}. In practice, the Fourier coefficients are computed separately for each component function, whereas the channels are mixed in Fourier space.

Graph neural operator layers. Discretizations of functions on more general point clouds can be interpreted as graphs with a given connectivity. A graph neural network aggregates messages over graph neighbourhoods

$$\mathbf{g}_j = \sum_{i \in \text{Neigh}_j} K_{ij}(\mathbf{f}_i, \mathbf{f}_j). \quad (14)$$

A graph neural operator replaces this index-defined aggregation by a quadrature approximation to an integral over a coordinate-defined neighbourhood $D(y_j)$

$$g(y_j) = \sum_{i: x_i \in D(y_j)} K(x_i, y_j, f(x_i), f(y_j)) \Delta_i \approx \int_{x \in D(y_j)} K(x, y_j, f(x), f(y_j)) dx. \quad (15)$$

Coordinates and quadrature weights are the key additions that make message passing consistent across meshes, point clouds and resolutions. Depending on the choice of $D(y_j)$, graph neural operators can represent local or global integral operators.

Attention-based operator layers. Transformers can be converted into neural operators using the same principle. A standard self-attention layer can be written as

$$\mathbf{g}_j = \sum_{i=1}^n \frac{K(\mathbf{f}_i, \mathbf{f}_j)}{\sum_{\ell=1}^n K(\mathbf{f}_\ell, \mathbf{f}_j)} \nu(\mathbf{f}_i), \quad (16)$$

where K is the attention kernel, and ν is the value map. For standard softmax attention, K is the positive kernel obtained after exponentiating a query–key score. For the operator version, let $\tilde{f}(y_j)$ denote the query-side feature: in self-attention on the same discretization, $\tilde{f}(y_j) = f(y_j)$; in cross-attention, it can be another function, a coordinate feature or a positional encoding on the output point cloud. The corresponding transformer neural operator layer is a global input-dependent integral operator

$$g(y_j) = \sum_{i=1}^n \frac{K(f(x_i), \tilde{f}(y_j))}{\sum_{\ell=1}^n K(f(x_\ell), \tilde{f}(y_j)) \Delta_\ell} \nu(f(x_i)) \Delta_i \approx \int_{D_f} \frac{K(f(x), \tilde{f}(y_j))}{\int_{D_f} K(f(z), \tilde{f}(y_j)) dz} \nu(f(x)) dx. \quad (17)$$

For equal quadrature weights, as on regular grids, the weights cancel and the formula reduces to standard attention. As the attention kernel acts on values of f , positional information must be supplied through the features, for instance, by concatenating coordinates or using positional encodings.

Encoder–decoder operator layers. A common technique in deep learning is to use encoders and decoders so that a learnable function, typically a neural network, operates in a latent space

$$\mathbf{g} = (\text{decoder} \circ \mathbf{K} \circ \text{encoder})(\mathbf{f}). \quad (18)$$

Such an idea can also be used to construct neural operator layers, but the encoder must be agnostic to the discretization of the input function and the decoder must produce a function that can be queried at arbitrary resolution. One representative construction encodes the input through inner products with functions b_j , applies a finite-dimensional map in latent space, and decodes at query coordinates

$$\mathbf{v}_j = \sum_{i=1}^n [b_j(x_i)]^* f(x_i) \Delta_i \approx \int_{D_j} [b_j(x)]^* f(x) dx, \quad (19)$$

$$\mathbf{w} = K(\mathbf{v}), g(y) = \text{decoder}(\mathbf{w}).$$

The latent dimension is fixed independently of the number of input samples, which makes the interface discretization agnostic. Encoder–decoder operators are universal in principle, but a fixed latent bottleneck can limit high-resolution information in practice.

Discretization, quadrature and interpolation

Numerical integration schemes are used to devise empirical versions of integral operator layers that can be applied to discretizations of functions at points $(x_i)_{i=1}^n$. These schemes typically take the form

$$\int_D H(x) dx \approx \sum_{i=1}^n H(x_i) \Delta_i, \quad (20)$$

for quadrature weights Δ_i and discretization points x_i . The quadrature weights Δ_i ensure that the sum approximates the integral representation and that the output is agnostic to the resolution. For instance, we can view the sum as a Riemann sum for the integral, which converges as we refine the discretization (Fig. 5). Different numerical integration schemes differ in their choices of weights and quadrature points, leading to different errors in the approximation for functions with certain regularity. This error dictates the discretization error of neural operators defined in terms of integral operators.

For regular grids, the quadrature weights are proportional to the product of the grid spacings in each dimension. For general point clouds, one can leverage a Delaunay triangulation to obtain a partition or use Monte Carlo integration when the points are randomly distributed according to a known density. For equal weights, the quadrature-aware statistics and losses used below reduce to the usual averages over the discretized vector. See Supplementary Section A.4 for more information.

Interpolation methods, such as spline, kernel or Fourier interpolations, allow transferring from a discrete to a continuous representation. They can be used after pointwise operators to allow querying at coordinates different from the input discretization and can also be applied to the input and output of a whole network to define a neural operator. This method is capable of converting any pretrained neural network into a neural operator, but fixing an input resolution can discard fine-scale details from high-resolution inputs and restrict the output function space by the choice of interpolation method.

The same function-space viewpoint also applies to training objectives and optimization: losses should be defined through discretization-consistent norms, and training strategies such as data augmentation, multiresolution training and autoregressive rollouts should preserve the operator interpretation whenever possible. Supplementary Section D.1 describes function-space losses, including Sobolev and physics-informed variants, whereas Supplementary Section D.2 describes data augmentation, multiresolution training, optimization and autoregressive training.

Architectures evaluated in the experiments

The evaluated architectures are U-Net, ViT, FNO, OFORMER, a kernel-interpolated U-Net, an encoder–decoder baseline based on fixed-grid interpolation and composite FNO variants with local convolutional components. The ordinary neural network baselines are U-Net and ViT, whose receptive fields or tokenization are tied to the training discretization. The neural operator architectures are FNO, OFORMER and the composite FNO variants, whereas the interpolated and encoder–decoder baselines are naive or partial operator constructions that isolate specific design choices.

The kernel-interpolated U-Net modifies the usual convolutional backbone by tying the convolutional kernel to the domain rather than to the pixel index stencil. The encoder–decoder baseline fixes a latent

grid, interpolates the input to that grid, applies a standard neural network and interpolates the result back to the requested query points. The composite FNO variants supplement spectral convolutions with local integral or differential operators, illustrating how neural operator components can be combined in a single architecture. Supplementary Section F.3 lists the architectural hyperparameters and implementation details, including the parameter regimes, learning-rate schedules, weight decay values, Fourier modes, hidden dimensions, attention heads and implementation sources used for each architecture.

Data, losses and training

The primary task uses the Navier–Stokes equations for incompressible fluids, where the model maps a forcing function to the vorticity at a later time from zero initial vorticity. We train the main models at a resolution of 128 and evaluate them across both lower and higher resolutions; in the mixed-resolution setting, models are trained on resolutions 64, 128, 256. In all cross-resolution evaluations, the same learned parameters are used at every test resolution, without resolution-specific retraining or fine-tuning and performance is reported as relative L^2 error on unseen test data. Supplementary Section F.1 gives the Navier–Stokes equations and additional model-size regimes. As a complementary benchmark, Supplementary Section F.2 describes the Darcy-flow experiments, which consider the steady-state two-dimensional Darcy equation and learn the map from pressure to forcing for a fixed diffusion coefficient, using padding for FNO-based architectures to handle non-periodic boundaries.

The loss function and metric are expressed on discretized functions through the same function-space quantities used for evaluation, with quadrature-aware norms used when appropriate. For instance, the squared L^2 loss between a prediction g and a ground truth function g^* is evaluated as

$$\text{Loss}(g, g^*) = \sum_{j=1}^m |g(y_j) - g^*(y_j)|^2 \Delta_j \approx \int_{D_g} |g(y) - g^*(y)|^2 dy. \quad (21)$$

When the governing equations are known, physics-informed residual losses formulated in function space can augment data losses for operator learning, as in physics-informed neural operators^{19,20}.

In our experiments, all models are trained on absolute L^2 error and evaluated using relative L^2 error. The experiments use Adam with model-specific learning-rate schedules and weight decay, and Supplementary Section F.3 details the computational setup and implementation sources. Multiresolution training is implemented by using each training sample at each selected resolution in each epoch.

Reporting summary

Further information on research design is available in the Nature Portfolio Reporting Summary linked to this article.

Data availability

Data are available via Zenodo at <https://doi.org/10.5281/zenodo.15687518> (ref. 21).

Code availability

We use the implementations of neural operators found in the NEURAL-OPERATOR¹⁴ Python library, publicly available via GitHub at github.com/neuraloperator/neuraloperator. Our code is available via Zenodo at <https://doi.org/10.5281/zenodo.20335280> (ref. 22).

References

1. Lagaris, I. E., Likas, A. & Fotiadis, D. I. Artificial neural networks for solving ordinary and partial differential equations. *IEEE Trans. Neural Netw.* **9**, 987–1000 (1998).
2. Raissi, M., Perdikaris, P. & Karniadakis, G. E. Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *J. Comput. Phys.* **378**, 686–707 (2019).

3. Mildenhall, B. et al. NeRF: representing scenes as neural radiance fields for view synthesis. In *Proc. 16th European Conference on Computer Vision* (eds Vedaldi, A. et al.) 405–421 (Springer International Publishing, 2020).
4. Sitzmann, V., Martel, J., Bergman, A., Lindell, D. & Wetzstein, G. Implicit neural representations with periodic activation functions. *Adv. Neural Inf. Process. Syst.* **33**, 7462–7473 (2020).
5. Jeong, Y. et al. PerFception: perception using radiance fields. In *Proc. Advances in Neural Information Processing Systems* Vol. 35 (eds Koyejo, S. et al.) 26105–26121 (NeurIPS, 2022).
6. Azizzadenesheli, K. et al. Neural operators for accelerating scientific simulations and design. *Nat. Rev. Phys.* **6**, 320–328 (2024).
7. Chen, P. Y. et al. CROM: continuous reduced-order modeling of PDEs using implicit neural representations. In *Proc. 11th International Conference on Learning Representations (ICLR)*, 2023).
8. Serrano, L. et al. Operator learning with neural fields: tackling PDEs on general geometries. In *Proc. Advances in Neural Information Processing Systems* Vol. 36 (eds Oh, A. et al.) 70581–70611 (NeurIPS, 2023).
9. Li, Z. et al. Geometry-informed neural operator for large-scale 3D PDEs. In *Proc. Advances in Neural Information Processing Systems* Vol. 36 (eds Oh, A. et al.) (NeurIPS, 2023).
10. Li, Z., Huang, D. Z., Liu, B. & Anandkumar, A. Fourier neural operator with learned deformations for PDEs on general geometries. *J. Mach. Learn. Res.* **24**, 1–26 (2023).
11. Rahman, M. A. et al. Pretraining codomain attention neural operators for solving multiphysics PDEs. In *Proc. Advances in Neural Information Processing Systems* Vol. 37 (eds Globersons, A. et al.) 104035–104064 (NeurIPS, 2024); <https://doi.org/10.52202/079017-3306>
12. George, R. J., Zhao, J., Kossaifi, J., Li, Z. & Anandkumar, A. Incremental spatial and spectral learning of neural operators for solving large-scale PDEs. *Trans. Mach. Learn. Res.* (2024).
13. Lanthaler, S., Stuart, A. M. & Trautner, M. Discretization error of Fourier neural operators. Preprint at <https://arxiv.org/abs/2405.02221> (2024).
14. Kossaifi, J. et al. A library for learning neural operators. Preprint at <https://arxiv.org/abs/2412.10354> (2024).
15. Li, Z. et al. Fourier neural operator for parametric partial differential equations. In *Proc. International Conference on Learning Representations (ICLR)*, 2021).
16. Duruisseaux, V., Kossaifi, J. & Anandkumar, A. Fourier neural operators explained: a practical perspective. Preprint at <https://arxiv.org/abs/2512.01421> (2025).
17. Li, Z., Meidani, K., Farimani, A. B. Transformer for partial differential equations' operator learning. *Trans. Mach. Learn. Res.* (2023).
18. Liu-Schiaffini M. et al. Neural operators with localized integral and differential kernels. In *Proc. 41st International Conference on Machine Learning* Vol. 235 (eds Salakhutdinov, R. et al.) 32576–32594 (PMLR, 2024); <https://proceedings.mlr.press/v235/liu-schiaffini24a.html>
19. Li, Z. et al. Physics-informed neural operator for learning partial differential equations. *ACM/IMS J. Data Sci.* **1**, 1–27 (2024).
20. Ganeshram, A. et al. FC-PINO: high precision physics-informed neural operators via Fourier continuation. Preprint at <https://arxiv.org/abs/2211.15960> (2025).
21. Berner, J. et al. Principled approaches for extending neural architectures to function spaces for operator learning (dataset). Zenodo <https://doi.org/10.5281/zenodo.15687518> (2025).
22. Berner, J. et al. neuraloperator/NNs-to-NOs: initial release: 1.0.0. Zenodo <https://doi.org/10.5281/zenodo.20335280> (2026).
23. Li, Z. et al. Neural operator: graph kernel network for partial differential equations. In *ICLR 2020 Workshop on Integration of Deep Neural Models and Differential Equations* (2020); <https://openreview.net/forum?id=fg2ZFmXFO3>
24. Rahman, M. A., Ross, Z. E. & Azizzadenesheli, K. U-NO U-shaped neural operators. *Trans. Mach. Learn. Res.* (2023).
25. Bonev, B. et al. Spherical Fourier neural operators: learning stable dynamics on the sphere. In *Proc. 40th International Conference on Machine Learning* Vol. 202 (eds Krause, A. et al.) 2806–2823 (PMLR, 2023).

Acknowledgements

We thank R. J. George for his input on the initial drafts. We thank Z. Li and M. A. Rahman for fruitful discussions.

Author contributions

J.B., M.L.-S., J.K. and V.D. led the development, organization and writing of the paper, and prepared the figures. M.L.-S. performed the coding, implementation and experimental work. J.B. established most of the initial draft. V.D. led the formatting, restructuring and reorganization of the paper for publication. B.B. provided substantial feedback, proofread the paper and contributed to selected subsections. K.A. provided feedback and proofread the paper. A.A. supervised the project, guided the overall organization of the paper, and provided feedback and proofreading.

Funding

M.L.-S. was supported in part by the Mellon Mays Undergraduate Fellowship. A.A. is supported in part by the Bren endowed chair, ONR (MURI grant no. N00014-18-12624) and by the AI2050 senior fellow programme at Schmidt Sciences.

Competing interests

The authors declare no competing interests.

Additional information

Supplementary information The online version contains supplementary material available at <https://doi.org/10.1038/s42256-026-01267-z>.

Correspondence and requests for materials should be addressed to Anima Anandkumar.

Peer review information *Nature Machine Intelligence* thanks Aniruddha Bora, Jonathan W. Siegel and the other, anonymous, reviewer(s) for their contribution to the peer review of this work.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

© The Author(s) 2026

Reporting Summary

Nature Portfolio wishes to improve the reproducibility of the work that we publish. This form provides structure for consistency and transparency in reporting. For further information on Nature Portfolio policies, see our [Editorial Policies](#) and the [Editorial Policy Checklist](#).

Statistics

For all statistical analyses, confirm that the following items are present in the figure legend, table legend, main text, or Methods section.

n/a	Confirmed
☒	☐ The exact sample size (<i>n</i>) for each experimental group/condition, given as a discrete number and unit of measurement
☒	☐ A statement on whether measurements were taken from distinct samples or whether the same sample was measured repeatedly
☒	☐ The statistical test(s) used AND whether they are one- or two-sided <i>Only common tests should be described solely by name; describe more complex techniques in the Methods section.</i>
☒	☐ A description of all covariates tested
☒	☐ A description of any assumptions or corrections, such as tests of normality and adjustment for multiple comparisons
☒	☐ A full description of the statistical parameters including central tendency (e.g. means) or other basic estimates (e.g. regression coefficient) AND variation (e.g. standard deviation) or associated estimates of uncertainty (e.g. confidence intervals)
☒	☐ For null hypothesis testing, the test statistic (e.g. <i>F</i> , <i>t</i> , <i>r</i>) with confidence intervals, effect sizes, degrees of freedom and <i>P</i> value noted <i>Give P values as exact values whenever suitable.</i>
☒	☐ For Bayesian analysis, information on the choice of priors and Markov chain Monte Carlo settings
☒	☐ For hierarchical and complex designs, identification of the appropriate level for tests and full reporting of outcomes
☒	☐ Estimates of effect sizes (e.g. Cohen's <i>d</i> , Pearson's <i>r</i>), indicating how they were calculated

Our web collection on [statistics for biologists](#) contains articles on many of the points above.

Software and code

Policy information about [availability of computer code](#)

Data collection	A pseudo-spectral scheme was used to solve the Navier-Stokes equations to generate our data. The specific details can be found in Section 4.1 of this paper: Kossaifi, Jean, et al. "Multi-grid tensorized fourier neural operator for high-resolution pdes." arXiv preprint arXiv:2310.00120 (2023).
Data analysis	Python libraries were used to analyze the data. Specifically, Matplotlib was used for visualization, and NumPy and PyTorch were used for manipulation and for training machine learning models, respectively.

For manuscripts utilizing custom algorithms or software that are central to the research but not yet described in published literature, software must be made available to editors and reviewers. We strongly encourage code deposition in a community repository (e.g. GitHub). See the Nature Portfolio [guidelines for submitting code & software](#) for further information.

Data

Policy information about [availability of data](#)

All manuscripts must include a [data availability statement](#). This statement should provide the following information, where applicable:

- Accession codes, unique identifiers, or web links for publicly available datasets
- A description of any restrictions on data availability
- For clinical datasets or third party data, please ensure that the statement adheres to our [policy](#)

Our data is fully available here: <https://zenodo.org/records/15687518>

Research involving human participants, their data, or biological material

Policy information about studies with [human participants or human data](#). See also policy information about [sex, gender \(identity/presentation\), and sexual orientation](#) and [race, ethnicity and racism](#).

Reporting on sex and gender

N/A

Reporting on race, ethnicity, or other socially relevant groupings

N/A

Population characteristics

N/A

Recruitment

N/A

Ethics oversight

N/A

Note that full information on the approval of the study protocol must also be provided in the manuscript.

Field-specific reporting

Please select the one below that is the best fit for your research. If you are not sure, read the appropriate sections before making your selection.

☐ Life sciences ☐ Behavioural & social sciences ☐ Ecological, evolutionary & environmental sciences

For a reference copy of the document with all sections, see [nature.com/documents/nr-reporting-summary-flat.pdf](https://www.nature.com/documents/nr-reporting-summary-flat.pdf)

Life sciences study design

All studies must disclose on these points even when the disclosure is negative.

Sample size

N/A

Data exclusions

N/A

Replication

N/A

Randomization

N/A

Blinding

N/A

Behavioural & social sciences study design

All studies must disclose on these points even when the disclosure is negative.

Study description

N/A

Research sample

N/A

Sampling strategy

N/A

Data collection

N/A

Timing

N/A

Data exclusions	N/A
Non-participation	N/A
Randomization	N/A

Ecological, evolutionary & environmental sciences study design

All studies must disclose on these points even when the disclosure is negative.

Study description	N/A
Research sample	N/A
Sampling strategy	N/A
Data collection	N/A
Timing and spatial scale	N/A
Data exclusions	N/A
Reproducibility	N/A
Randomization	N/A
Blinding	N/A

Did the study involve field work? ☐ Yes ☒ No

Reporting for specific materials, systems and methods

We require information from authors about some types of materials, experimental systems and methods used in many studies. Here, indicate whether each material, system or method listed is relevant to your study. If you are not sure if a list item applies to your research, read the appropriate section before selecting a response.

Materials & experimental systems

Methods

n/a	Involved in the study
☒	☐ Antibodies
☒	☐ Eukaryotic cell lines
☒	☐ Palaeontology and archaeology
☒	☐ Animals and other organisms
☒	☐ Clinical data
☒	☐ Dual use research of concern
☒	☐ Plants

n/a	Involved in the study
☒	☐ ChIP-seq
☒	☐ Flow cytometry
☒	☐ MRI-based neuroimaging

Plants

Seed stocks	N/A
Novel plant genotypes	N/A
Authentication	N/A